Simplification frameworks

Arend's `simplify` meta versus Lean 4's `simp` and simprocs,
with a design study for `field_simp` in Arend

Abstract

This note compares the simplification infrastructure in `arend-lib` (`meta/src/main/java/org/arend/lib/meta/simplify/`) with Lean 4's `simp` engine and its simproc extension mechanism. Section 1 documents the Arend framework as implemented; Section 2 does the same for Lean; Section 3 contrasts them along the axes that actually differ. Section 4 treats extensibility precisely: both systems are fully open to third-party tactic authors, and the real distinction is the *granularity* of the extension unit rather than who is permitted to extend. Section 5 is a case study of `field_simp`, whose Lean 3 implementation was a rewrite-rule set and whose current Mathlib implementation is a normalising procedure exposed through simprocs [21, 23, 22]. Section 6 derives a cost model for implementing an analogous tactic as an Arend `LocalSimplificationRuleBase` subclass, and Section 7 recommends the reflective alternative. Section 8 records two incidental defects found while reading the code.

All claims about the Arend side are cited to file and line in this repository. Claims about the Lean side are linked inline to primary Lean and Mathlib documentation or source files listed in the bibliography. Where a statement is an estimate rather than a measurement, it is marked as such.

Contents

1 The Arend `simplify` framework **2**
- 1.1 Inventory . . . 2
- 1.2 Entry point and the two modes . . . 3
- 1.3 The traversal engine . . . 3
- 1.4 The rule interface . . . 4
- 1.5 Two rule families . . . 5
- 1.6 Instance-driven rule selection . . . 6
- 1.7 Dependency resolution . . . 6

2 Lean 4: `simp` and simprocs **6**
- 2.1 Attributes and environment extensions . . . 7
- 2.2 Discrimination trees . . . 7
- 2.3 The simp engine . . . 10
- 2.4 Congruence lemmas . . . 10
- 2.5 Simprocs . . . 12
- 2.6 Dischargers . . . 12

3 Systematic comparison **12**

4 Extensibility, stated precisely **13**
- 4.1 Anyone can ship tactics . . . 13
- 4.2 But this tactic's rule table is closed . . . 14
- 4.3 The distinction that matters: granularity . . . 14
- 4.4 Arend already has the attribute primitive . . . 14

5 Case study: `field_simp` 16
5.1 The first design: a simp set plus a discharger . . . 16
5.2 The new normaliser and its `simp [field]` front end . . . 17
5.3 The discharger . . . 19
5.4 Two front ends over one core . . . 20

6 Cost model for a local-rule implementation in Arend 20
6.1 Per-step cost . . . 20
6.2 Why this is fine today and would not be for `field_simp` . . . 21
6.3 Estimated knee . . . 21
6.4 The blocker is not performance . . . 21
6.5 The same hole, seen from the congruence side . . . 21

7 Recommended design 22
7.1 Cheap way to falsify the estimate first . . . 22

8 Incidental defects observed 23

9 Summary 23

10 Repository source locations 23

References 24

1 The Arend `simplify` framework

1.1 Inventory

The framework is thirteen Java files, roughly 1200 lines, under `arend-lib/meta/src/main/java/org/arend/lib/meta/simplify/`.

File	Role	Lines
`SimplifyMeta.java`	meta entry point, dependency refs	105
`Simplifier.java`	traversal, motive construction	308
`SimplificationRule.java`	the extension interface	14
`LocalSimplificationRuleBase.java`	base for local rewrite rules	131
`GroupRuleBase.java`	base for reflective group rules	58
`MonoidIdentityRule.java`	$x \cdot 1 \to x$, $1 \cdot x \to x$	63
`MultiplicationByZeroRule.java`	$x \cdot 0 \to 0$	40
`MulOfNegativesRule.java`	$(-x)(-y) \to xy$	56
`DoubleNegationRule.java`	$-(-x) \to x$	45
`IdentityInverseRule.java`	$1^{-1} \to 1$	44
`NegationPropagationRule.java`	$-(x \cdot y) \to (-x) \cdot y$	53
`GroupInverseRule.java`	reflective group normaliser	165
`AbGroupInverseRule.java`	reflective abelian-group normaliser	118

It is registered as a single tactic, `Algebra.simplify`, at `StdExtension.java:537`:

```
contributor.declare(
  text("Simplifies the expected type or the type of the argument if the expected type is
    unknown."),
  makeDef(algebra, "simplify",
    new DependencyMetaTypechecker(SimplifyMeta.class,
      () -> new DeferredMetaDefinition(new SimplifyMeta(), true))));
```

The `DeferredMetaDefinition` wrapper defers invocation until the expected type has been solved, which the implementation depends on: it branches on whether `contextData.getExpectedType()` is null.

1.2 Entry point and the two modes

`SimplifyMeta.java:77` distinguishes a *goal* mode from a *forward* mode:

```
boolean isForward = contextData.getExpectedType() == null;
...
var expression = args.isEmpty()
  ? factory.ref(typechecker.getPrelude().getIdpRef())
  : args.getFirst().getExpression();
if (isForward) {
  var checkedExpr = typechecker.typecheck(expression, null);
  type = checkedExpr == null ? null : checkedExpr.getType();
} else {
  type = expectedType == null ? null : expectedType.getUnderlyingExpression();
}
```

Three consequences worth stating explicitly:

1. With no argument the tactic substitutes `idp` (`SimplifyMeta.java:88`). So a bare `simplify` closes the goal precisely when simplification renders both sides of the equation syntactically identical. This is why `test/Algebra/SimplifyTest.ard` can write `\func test1 {M : Monoid} (x : M) : x = x * ide => simplify`.
2. In forward mode the argument's type is simplified rather than the goal; combined with the `in` meta this yields `simplify in t`, the analogue of Lean's `simp at h` [4].
3. `simplify` does not transform a goal in place. It returns a term of the *original* type, built by transporting along the collected equalities. The tactic is a term-level transport, not a goal rewriter.

1.3 The traversal engine

`Simplifier.java` contains the engine. `simplifyTypeOfExpression` (line 227) normalises the type to WHNF, runs an inner-class visitor over its subexpressions, and collects a list of pairs (occurrence, proof):

```
CoreExpression normType = type.normalize(NormalizationMode.WHNF);
var processor = new SimplifyExpressionProcessor();
typechecker.withCurrentState(tc -> normType.processSubexpression(processor));
var occurrences = processor.getSimplificationOccurrences()
                    .stream().map(x -> x.proj1).collect(Collectors.toList());
if (occurrences.isEmpty()) {
  errorReporter.report(new TypecheckingError("Nothing to simplify", refExpr));
  return expression;
}
```

It then abstracts every occurrence into a fresh variable `y0 ...yn`, builds a motive lambda whose body is produced by a nested meta that performs the substitution via `replaceSubexpressions`, and finally threads the equalities through a chain of transports (line 305):

```
return RewriteEquationMeta.chainOfTransports(
  factory.ref(meta.transport, refExpr.getLevels()),
  checkedLam.getExpression(), proofs, expression, factory, false);
```

This is the framework's answer to congruence. Lean rewrites inside subterms using generated and user-supplied `@[congr]` lemmas [12, 13]; Arend instead performs higher-order abstraction of

the rewrite sites and transports along the motive. No congruence lemma database is required, which is a genuine simplification, at the cost of building a motive whose size is proportional to the whole type.

Locally, within a single rule application, congruence is handled cubically rather than by transport — `LocalSimplificationRuleBase.java:96--124` constructs `\lam i => ... p @ i ...` and wraps it in `path`:

```
var ideElimPath = factory.at(simplifyRes.proj2, factory.core(checkedVar));
...
var exprPath = factory.path(factory.core(checkedLam));
```

This is not an idiom local to the simplifier. It is what congruence *is* in Arend, spelled out at `arend-lib/src/Paths.ard:3`:

```
\func pmap {A B : \Type} (f : A -> B) {a a' : A} (p : a = a') : f a = f a'
  => path (\lam i => f (p @ i))
```

Because `=` is a path type rather than an inductive family with a single constructor, a proof `p : a = a'` is a function from the interval. Substituting `p @ i` into any context, abstracting `i` and wrapping in `path` yields a proof that the two instantiations are equal. No lemma database, no per-function generation, no classification of arguments: congruence follows from what equality is. `LocalSimplificationRuleBase` inlines this rather than calling `pmap` only because its context is an arbitrary subterm position rather than a named `f`.

What this costs. The uniformity has one hole, and it is already visible in the code. Abstracting an occurrence can break the type of something that depended on it, and there is no vocabulary for repairing that — so the motive simply fails to typecheck (`Simplifier.java:292`):

```
errorReporter.report(typeWithOccur == null
  ? new SimplifyError(typechecker.getExpressionPrettifier(), occurrences, normType, refExpr)
  : new TypeError(typechecker.getExpressionPrettifier(),
      "Cannot substitute a variable. The resulting type is invalid", typeWithOccur, refExpr))
    ;
```

That error is dependent-congruence failure. Section 2.4 describes the explicit heterogeneous, cast and subsingleton argument classifications Lean uses to reconcile such dependencies [12].

1.4 The rule interface

The entire extension surface is fourteen lines (`SimplificationRule.java`):

```
public interface SimplificationRule {
  /**
   * @return Pair (simplifiedExpr, p : simplifiedExpr = expression).
   *         Null if expression cannot be simplified.
   */
  RewriteEquationMeta.EqProofConcrete apply(TypedExpression expression);

  ConcreteExpression finalizeEqProof(ConcreteExpression proof);
}
```

Note what the signature *cannot* express. There is no channel by which a rule can report "I can rewrite this provided someone proves $b \neq 0$". There is no discharger parameter, no subgoal output, no discharge-depth budget. This is the decisive constraint for Section 5.

Note also that `apply` returns `EqProofConcrete`, whose fields are *concrete* (surface-syntax) expressions. Rules therefore emit syntax that the engine re-elaborates, rather than emitting core terms directly. See Section 6.

1.5 Two rule families

Local rewrite rules. Six rules extend `LocalSimplificationRuleBase` and implement

```
protected abstract Pair<CoreExpression, ConcreteExpression>
  simplifySubexpression(CoreExpression subexpr);
```

The base class locates a subexpression whose type matches the ambient carrier, applies the hook, builds the cubical congruence proof, and iterates to a fixed point. `MonoidIdentityRule` is representative:

```
List<CoreExpression> args = mulMatcher.match(subexpr);
if (args != null) {
  var left = args.get(args.size() - 2);
  var right = args.getLast();
  boolean isIdeOnTheLeft = ideMatcher.match(left) != null;
  var ide = isIdeOnTheLeft ? left
          : ideMatcher.match(right) != null ? right : null;
  if (ide != null) {
    CoreExpression value = ide == left ? right : left;
    return new Pair<>(value, factory.appBuilder(
        factory.ref(isIdeOnTheLeft ? ideLeft : ideRight))
      .app(factory.hole(), false)
      .app(factory.core(value.computeTyped()), false).build());
  }
}
return null;
```

Reflective normalisers. `GroupInverseRule` and `AbGroupInverseRule` are not rewrite rules at all. They compile the expression into a term AST, normalise it in Java, and emit a single application of a correctness lemma proved in Arend:

```
var term = CompiledTerm.compile(expression.getExpression(),
             Arrays.asList(mulMatcher, invMatcher, ideMatcher), values);
var newTerm = simplify(term);
if (newTerm == null) return null;
...
var simplifyProof = factory.appBuilder(factory.ref(meta.simplifyCorrectInv))
  .app(factory.ref(dataRef), false).app(concreteTerm).build();
```

The Arend-side counterpart lives in `arend-lib/src/Algebra/Group/Solver.ard`:

```
\func simplify (t : GroupTerm V) : GroupTerm V
   => \case find-fstLeafToRemoveInd t, isInNF-dec t \with {
        | just (ind, p, _, _), just q => (remove-pair t ind).1
        | _, _ => t
     }

\func simplify-correct (t : GroupTerm V) : interpret t = interpret (simplify t)
   => unfold simplify (mcases {1} \with {
     | just (ind, p, lb, ub), just q => inv $ rp-correctness-both-inside t q ind p lb ub
     | _, _ => idp
   })
```

(`Solver.ard:504` and `Solver.ard:507`; the abelian variant is at `Solver.ard:576`.) This is proof by reflection: one proof term regardless of how many cancellations the normaliser performs. It is structurally Lean's `ring`, not Lean's `simp`. The cost is that the normaliser exists twice — once in Java, once in Arend — with nothing enforcing agreement. The comparison with `ring` refers to Mathlib's proof-producing normaliser, which likewise evaluates a reified normal form [28].

1.6 Instance-driven rule selection

Rule lookup is the sharpest departure from Lean. `simplify` does not index rules by head symbol. It performs *instance search on the type of the subterm* and then selects rules by a cascade of subclass tests (`Simplifier.java:150--205`):

```
var possibleClasses = new HashSet<>(Arrays.asList(meta.Monoid, meta.AddMonoid,
  meta.Semiring, meta.Ring, meta.AddGroup, meta.Group, meta.CGroup, meta.AbGroup));
var instanceClassCallPair = Utils.findInstanceWithClassCall(
  new InstanceSearchParameters() {
    @Override public boolean testClass(@NotNull CoreClassDefinition classDefinition) {
      for (var clazz : possibleClasses)
        if (classDefinition.isSubClassOf(clazz)) return true;
      return false;
    }
  }, meta.carrier, type, typechecker, refExpr, null);
if (instanceClassCallPair != null) {
  ...
  if (classCall.getDefinition().isSubClassOf(meta.Monoid))
    rules.add(new MonoidIdentityRule(instance, classCall, meta, refExpr, typechecker, false))
    ;
  if (classCall.getDefinition().isSubClassOf(meta.AddMonoid))
    rules.add(new MonoidIdentityRule(instance, classCall, meta, refExpr, typechecker, true));
  ...
}
```

There is a good reason for this design. In Arend, `x * y` elaborates to `Semigroup.* {M} x y`: a class-field projection applied through an instance. Matching therefore has to be instance-aware, which is why every rule is built from `FunctionMatcher.makeFieldMatcher(classCall, instance, meta.mul, ...)`. Lean faces the same problem with `HMul.hMul` and solves it with discrimination-tree keys that replace instance arguments by wildcards and check candidates later by unification [9]. Arend's approach resolves the instance once per subterm type and is arguably more precise; the price is an instance search per visited node, and the impossibility of keying rules on head symbol at all.

1.7 Dependency resolution

`SimplifyMeta` declares the lemmas it needs by qualified name, resolved declaratively at extension load:

```
@Dependency(name = "AddGroup.negative-isInv")     ArendRef negIsInv;
@Dependency(name = "Group.inverse-isInv")         ArendRef invIsInv;
@Dependency(name = "Monoid.ide-left")             ArendRef ideLeft;
@Dependency(name = "PseudoSemiring.zro_*-right")  ArendRef zeroMulRight;
@Dependency(name = "CGroupData.simplify-correct") ArendRef simplifyCorrectAbInv;
```

This mechanism is nicer than the raw `Name`-based lookups visible in Lean metaprograms such as the current `field_simp` implementation [22]: resolution failures surface at load time rather than at tactic invocation. It carries over unchanged to any redesign.

2 Lean 4: `simp` and simprocs

Unless a release note or language-reference page is cited explicitly, Lean and Mathlib implementation links in this note are pinned to their respective `v4.32.1` revisions so that line-level implementation claims remain reproducible.

2.1 Attributes and environment extensions

Because the rest of this note leans on them, it is worth stating what a Lean attribute actually is. An attribute is metadata attached to a declaration, written either at declaration time or afterwards [1]:

```
@[simp] theorem foo : a * 1 = a := ...   -- at declaration
attribute [simp] foo                      -- after the fact
```

What makes `simp` more than decoration is that its attribute is backed by an *environment extension*: a table serialised into the compiled `.olean` and merged on import. So `@[simp] theorem foo` does not annotate `foo`; it *inserts `foo` into a database*. That is why `simp` can fire a lemma tagged three imports away without anyone naming it. Three moving parts: a syntactic marker, a hook that runs when the declaration is processed, and a persistent import-merged table [1, 2, 5].

Attributes are not a fixed built-in list. They are declared, and the macro form is sugar over the same `initialize` idiom used by `Mathlib/Tactic/FieldSimp/Attr.lean` [5, 15, 24]:

```
register_simp_attr myLemmas "my own simp attribute"
initialize myExt : SimpExtension <- registerSimpAttr `myLemmas "my own simp attribute"
```

Both create an attribute one writes as `@[myLemmas]`; the second also yields the extension handle, for reading the set programmatically. Two practical consequences: a lemma tagged `@[myLemmas]` is *not* in the default simp set and must be requested explicitly as `simp [myLemmas]`, which is the point of a custom set — a library ships domain-specific rewriting without slowing everyone else's `simp`; and a custom simp attribute cannot be used in the file that declares it, which is why Mathlib centralises its declarations in `Mathlib/Tactic/Attr/Register.lean` [5, 7, 27].

The two flavours of set differ in what lands in the table:

- A **lemma set** (`SimpExtension`, holding `SimpTheorems`) stores the *statement* of a theorem, turned into a rewrite rule keyed on its left-hand-side head pattern. Declarative: you write an equation and the engine applies it.
- A **simproc set** (`Simp.SimprocExtension`, holding `Simprocs`) stores a compiled `Simp.Simproc` function, keyed on a declared pattern. Executable: it can compute, inspect, call other tactics, or decline.

These two representations are the structures exposed by Lean's simp-theorem and simproc APIs [11, 15].
A lemma suffices when the rewrite *is* one equation. A procedure is needed when the result depends on computation — which is precisely why `field_simp` is a procedure (Section 5): no fixed set of equations directly expresses the normalisation algorithm used by the current implementation [22].

The two flavours are paired by default. Since Lean 4.6.0, `register_simp_attr X` creates *both* a lemma set and a simproc set named `X`, so that `simp [X]` retrieves both and the user need not know they are stored in different structures. Pairing is therefore the normal case, obtained for free. This matters when reading Section 5: an attribute that has *no* lemma half has actively declined the default rather than merely omitted something. (A related asymmetry: `simp only` does not consult the default simproc set, though individual simprocs may be named explicitly, as in `simp only [reduceFoo]`.) Both behaviours were introduced and illustrated in the Lean 4.6.0 release notes and remain explicit in the tactic elaborator [8, 18].

2.2 Discrimination trees

`simp` may carry many lemmas. On reaching a subterm it needs the handful whose left-hand side could possibly match; trying all of them is hopeless. A discrimination tree is that filter, and

since the rest of this note speaks of rules being "keyed" on patterns, it is worth defining. The details below follow Lean's discrimination-tree module documentation [9, 10].

Head symbol. In an application `f a b c`, the *head* is `f`. A lemma's left-hand side has one too: `?x * 1 = ?x` has head `HMul.hMul`. Indexing *on the head pattern* means the first discriminating key is the head symbol, so a lemma about multiplication is consulted only where `simp` is standing on a multiplication [11, 9].

A trie stores sequences. An ordinary lookup table maps a key to a value. A trie stores *sequences* as paths — one element per edge — so that sequences sharing a prefix share a path:

```
root
 |
 +-- c -- a --+-- r        -> "car"
 |            |
 |            +-- t        -> "cat"
 |
 +-- d -- o -- g           -> "dog"
```

"cat" is found by walking `c`, `a`, `t`; the `dog` branch is never touched. Early path elements can therefore eliminate most of the tree; prefix filtering is the principal source of a trie's lookup speed.

A term is a tree, so flatten it into a sequence. `f (g a) b` is the tree

```
    f
   / \
  g   b
  |
  a
```

Walk it in *preorder* — node first, then children left to right — recording each node's number of children:

visit	node	children
1	`f`	2
2	`g`	1
3	`a`	0
4	`b`	0

which is the sequence `[<f,2>, <g,1>, <a,0>, <b,0>]`. The child counts make this reversible: `<f,2>` announces that two complete subtrees follow, `<g,1>` that one follows, and `<a,0>` that none does — so `g`'s subtree ends there and `<b,0>` must be `f`'s second child. The flat sequence and the tree carry the same information.

Several lemmas in one trie. Take three lemmas, suppressing the type and instance arguments that the next paragraph explains are erased anyway:

lemma	left-hand side	key sequence
`add_zero`	`?x + 0`	`[<Add,2>, *, <0,0>]`
`zero_add`	`0 + ?x`	`[<Add,2>, <0,0>, *]`
`mul_one`	`?x * 1`	`[<Mul,2>, *, <1,0>]`

A lemma's variables become `star` in Lean's index [9]. Filed together:

```
root
 |
 +-- <Add,2> --+-- *        -- <0,0>     -> add_zero
 |             |
 |             +-- <0,0>    -- *         -> zero_add
 |
 +-- <Mul,2> --+-- *        -- <1,0>     -> mul_one
```

Standing on a multiplication, `simp` takes the `<Mul,2>` branch and both `Add` lemmas are eliminated in a single step. That is what "keyed on the head symbol" buys.

A lookup, walked. Query `a + 0`, i.e. `[<Add,2>, <a,0>, <0,0>]`:

1. `<Add,2>` — take that branch.
2. `<a,0>` — no literal `<a,0>` child exists, but there is a `*` child; follow it, consuming the query's subterm `a`.
3. `<0,0>` — matches. `add_zero` is a candidate.

Now query `0 + 0`, i.e. `[<Add,2>, <0,0>, <0,0>]`. At position 2 *both* children match — the `*` and the literal `<0,0>` — so retrieval follows both, returning `add_zero` via the first and `zero_add` via the second. Both genuinely apply.

This is why the result is a *superset* of the matching lemmas: the tree returns candidates and real unification sorts them out afterwards. Lean's own source header calls these "(Imperfect) discrimination trees" for exactly this reason. The tree is a cheap filter, never a decision procedure [9].

Why arity is actually needed. Query `(a * b) + 0`:

```
[<Add,2>, <Mul,2>, <a,0>, <b,0>, <0,0>]
```

Take the `<Add,2>` branch. Position 2 is `<Mul,2>`, which has no literal child, so follow `*` — and now the question is how much of the query that wildcard swallows. The subterm is `a * b`, occupying three keys: `<Mul,2>, <a,0>, <b,0>`. Skip exactly those, and the next key is `<0,0>`, giving `add_zero`.

Arity is what lets one compute "three". A node of arity n is followed by n complete subtrees, so a running count locates the end of any subterm. Without arities the sequence would be `[Add, Mul, a, b, 0]` and nothing would say where the wildcard's match stops. That is the *skip* operation the source names. The second reason is cheaper to state: `<f,1>`, `<f,2>` and `<f,3>` distinguish `f a`, `f a b` and `f a b c`, which a bare `f` would conflate [9].

The key constructors. Only now is the list meaningful — it is the alphabet these sequences are written in:

Constructor	Stands for
`const name arity`	a constant applied to *arity* arguments
`fvar id arity`	a local hypothesis, likewise applied
`lit`	a numeric or string literal
`star`	wildcard: a lemma variable, or a term deliberately ignored
`arrow`	an implication or function arrow
`proj`	a structure projection
`other`	anything else — nested lambda, sort, and so on

This is the current `DiscrTree.Key` datatype [10].

What the key omits, and why. This is the clause that matters for Section 3. An addition is not a bare application: `a + b` elaborates to `Add.add` α `inst a b`. The stored paths look like

```
<Add.add, 4>, α, *, *, *            -- general
<Add.add, 4>, Nat, *, <a,0>, <b,0>  -- at Nat
```

with the instance slot a `star`. The source's justification is that instances are "morally" canonical but not syntactically unique — `Ring.hasAdd Int Int.isRing` and `Int.hasAdd` may be definitionally equal yet distinct terms. Were the instance part of the key, a lemma stated for an abstract monoid would never be retrieved at `2 * 3 : Nat`. So it is erased from the index and checked later, during unification.

Also omitted: proofs, by proof irrelevance, and universe levels. Implicit arguments are omitted only when they are *not* types — dropping the implicit `Nat` in `Decidable (@Eq Nat x y)` would place every decidable-equality instance on a single path [9].

One caveat worth recording, since it is the mirror image of Arend's situation: keys are computed at `TransparencyMode.reducible`, and class projections are **not** reducible. `Add.add Nat inst a b` is therefore not folded into `Nat.add a b`; the `Add.add` form is the de-facto canonical one, and re-packing is the metaprogrammer's responsibility [9].

The same obstacle, the opposite solution. Both systems meet the identical problem — `x * y` is not an application of `*` but a projection through an instance — and answer it in opposite directions. Lean asks *"what is the head symbol here?"*, having erased the instance from the key so that one lemma serves every instantiation. Arend asks *"what structure is this term's type an instance of?"*, resolving the instance first with `findInstanceWithClassCall` and then matching fields against it with `makeFieldMatcher` (Section 1) [9].

The consequence is a difference in how lookup scales. Lean's is a trie walk, substantially independent of how many lemmas exist. Arend's is an instance search per visited node, and cannot key on head symbol at all — which is why adding rules to `simplify` does not scale the way adding `@[simp]` lemmas does. This scaling conclusion is an inference from the two lookup algorithms rather than a benchmark; the Lean half is documented in [9, 11].

2.3 The simp engine

Lean's `simp` is a rewriting engine over a database of simp theorems (`SimpTheorems`), populated by the `@[simp]` attribute and by scoped/named sets created with `register_simp_attr`. Candidate lookup is by discrimination tree (Section 2.2). Rewriting descends into subterms using congruence lemmas (`@[congr]`), handles binders and dependent congruence, and produces `Eq`, `Iff`, or `HEq` proofs. Traversal has `pre` and `post` phases, results are cached per call, and termination is bounded by `maxSteps` (default 100000) and `maxDischargeDepth` [3, 11, 16, 6].

2.4 Congruence lemmas

Having rewritten a subterm, `simp` must lift the result: from `a = a'` it needs `f x a y = f x a' y`. The naive tool, `congrArg`, covers the easy case and fails in four others:

1. **Dependent functions.** If `f : (a :` α`) →` β `a` then `f` a_1 and `f` a_2 inhabit *different types*, so `Eq` does not typecheck and `HEq` is required.
2. **Proof arguments.** For `f : (a :` α`) → p a →` β, rewriting `a` changes the type of the proof argument, though proof irrelevance means nothing should really need doing.
3. **Instance arguments.** `ite b x y` carries `[Decidable b]`; rewriting `b` to `c` obliges the simplifier to recompute the instance.

4. **Contextual information.** When simplifying the `then` branch of `if c then x else y` one wants `c` in scope as a hypothesis. Plain congruence offers nowhere to put it.

Lean's generated congruence-theorem API documents the dependent, proof and instance cases, while its hand-written simp-congruence table supplies the contextual cases [12, 13].
A congruence lemma answers all four by classifying how each argument participates. Lean's vocabulary for that classification is `CongrArgKind`, with six constructors:

Kind	Meaning
`fixed`	the argument is identical on both sides; no rewriting
`fixedNoParam`	not a parameter at all — the theorem was specialised for it (a subsingleton other parameters depend on)
`eq`	introduces `a_i`, `b_i` and `eq_i : a_i = b_i`
`heq`	as `eq`, but heterogeneous
`cast`	the argument is a subsingleton/proposition determined by earlier arguments
`subsingletonInst`	an instance argument, e.g. two `[Decidable ...]` parameters reconciled by subsingleton elimination

A `CongrTheorem` is then `{ type : Expr, proof : Expr, argKinds : Array CongrArgKind }` [12].

Mostly generated, occasionally written. `mkCongrSimp?` and `mkHCongr` synthesise congruence theorems per function, `getCongrSimpKinds` computes the argument kinds, and `congrKindsExt` — an environment extension, in the sense of Section 2.1 — stores argument-kind specialisations. The `@[congr]` attribute supplies user-defined simp-congruence theorems for cases where generic generation is insufficient. The canonical case is `ite`, whose hand-written lemma has the shape

```
(h₁ : b = c) (h₂ : c → x = u) (h₃ : ¬c → y = v) : ite b x y = ite c u v
```

Observe `h₂ : c → x = u`: the branch condition is available as a hypothesis while proving the branch equal. That is item 4, and no generic scheme can produce it, because nothing in the type of `ite` records that its second argument is reached only when the first is true [13, 14].

Against Arend. Of the four problems, `pmap` (Section 1) disposes of the first three uniformly and for free — substitution into a context is congruence when equality is a path type. What it has no vocabulary for is the dependent repair that `heq`, `cast` and `subsingletonInst` encode, which is why `Simplifier.java:292` reports *"Cannot substitute a variable. The resulting type is invalid"* where Lean has explicit dependent-reconciliation argument kinds [12]. Item 4 has no Arend analogue at all; see Section 6.5.

So the trade is not that one system has congruence and the other lacks it. Arend obtains congruence definitionally and pays at the margins; Lean pays for a generation-and-caching subsystem and buys graceful handling of the hard cases plus a place to thread contextual facts [12, 13, 14].

A disambiguation. `arend-lib`'s `cong` meta (`meta/src/main/java/org/arend/lib/meta/cong/CongruenceMeta.java`, declared as "proves an equality by congruence closure of equalities in the context") is unrelated. That is congruence *closure*, the analogue of Lean's `cc`, deriving `f a = g b` from `f = g` and `a = b`. It has nothing to do with `@[congr]` [19].

2.5 Simprocs

A simproc is a metaprogram registered against a discrimination-tree pattern, so it is invoked only at subterms matching that pattern. It is declared with `simproc` / `simproc_decl` / `builtin_simproc`, or `dsimproc` for definitional rewriting, and runs in `SimpM` with full `MetaM` access [15, 16]. An ordinary `Simproc` returns

```
Simp.Step = .done | .visit | .continue
```

each carrying a `Simp.Result` with fields `expr`, `proof?` and `cache`. A `DSimproc` instead returns the parallel `Simp.DStep` (an abbreviation for `TransformStep`): its three control constructors carry an `Expr` directly, and the output must be definitionally equal to the input. Two features have no Arend analogue:

- `proof? = none` asserts the step is definitional, so no proof term is built at all; this avoids constructing an equality proof for that step.
- The result is an elaborated `Expr`. Nothing is re-elaborated.

The `Result`, `Step`, `Simproc` and `DSimproc` definitions give these semantics directly [16].

2.6 Dischargers

Conditional simp lemmas produce side goals. `simp` discharges them with a `Simp.Discharge`, i.e. a function `Expr → SimpM (Option Expr)`, which may recurse into `simp` itself under `Simp.withIncDischargeDepth`. Custom dischargers are supplied as `simp (disch := tac)`. This is the mechanism that makes conditional rewriting practical, and it is the feature whose absence in Arend Section 5 turns on [17, 16, 18].

3 Systematic comparison

	Arend `simplify`	**Lean** `simp` + **simprocs**
Rule indexing	Instance search on the subterm's *type*, then an `isSubClassOf` cascade (`Simplifier.java:150`). Cost: one instance search per visited node.	Trie over the flattened term, keyed on the *head symbol*, with instance arguments erased to wildcards (Section 2.2). Cost: a trie walk, largely independent of rule count [9, 11].
Rule count	8 rules, all algebraic identities.	A global `@[simp]` database plus registered simproc sets [5, 15].
Congruence	*Definitional.* `=` is a path type, so substituting `p @ i` into a context and abstracting `i` already *is* the proof (`pmap`, `Paths.ard:3`). Inlined locally; `transport` at the root (`Simplifier.java:305`).	*Generated theorems*, one per function (`mkCongrSimp?`, `mkHCongr`), with argument-kind specialisations stored in `congrKindsExt`; `@[congr]` supplies user-defined simp-congruence theorems. Arguments are classified by `CongrArgKind` (Section 2.4) [12, 13].

	Arend `simplify`	**Lean** `simp` + **simprocs**
Dependent congruence	No vocabulary for it: the motive is built and either typechecks or does not (`Simplifier.java:292`).	`heq` / `cast` / `subsingletonInst` kinds encode the heterogeneous, cast and subsingleton reconciliation needed by dependent arguments [12].
Contextual rewriting	None. Neither mechanism has a slot for a positionally available hypothesis (Section 6.5).	`contextual := true`; hand-written `@[congr]` lemmas such as `ite`'s put the branch condition in scope [6, 14].
Proof-free steps	None. Every rule must return a path. Definitional work is delegated to `normalize(ENF)` before matching.	`proof? = none` marks a reflexive/definitional step; `dsimp` and dsimprocs are a first-class mode [16].
Rule output	*Concrete* syntax, re-elaborated by `typechecker.typecheck` (`Simplifier.java:117`).	Elaborated `Expr` plus proof `Expr` [16].
Side conditions	No channel. `SimplificationRule.apply` cannot emit a subgoal.	`discharge?` can recursively run simp; depth-bounded [17, 6].
Rule ordering	`TreeSet` ordered by `hashCode` difference (`Simplifier.java:78`); see Section 8.	Explicit priorities, with tie-breaking defined by the registered theorem order [5, 11].
Termination	`checkCancelled()` only; no step budget. Safe only because every existing rule strictly contracts the term.	`maxSteps`, `maxDischargeDepth`, configurable [6].
Caching	None.	Per-call expression cache; `cache` flag per result [16].
Invocation	`simplify`, `simplify p`, `simplify in t`; composable with `repeat`, `orElse`, `using`.	`simp only [...]`, `simp [-lem]`, `simp at h ⊢`, `simp?`, named sets [4, 5].

4 Extensibility, stated precisely

4.1 Anyone can ship tactics

It would be wrong to describe Arend's tactic layer as closed. Any Arend library can ship an extension. The library manifest names the jar directory and main class — `arend-lib/arend.yaml`:

```
langVersion: 1.12
version: 1.12
sourcesDir: src
extensionsDir: ext
extensionMainClass: org.arend.lib.StdExtension
```

Metas are contributed through `ArendExtension.declareDefinitions`, and a library obtains its dependencies' extension objects through `setDependencies` (`api/src/main/java/org/arend/ext/`

ArendExtension.java:31, invoked at base/src/main/java/org/arend/server/impl/LibraryService.java:158). Extension jars share a classloader tier (LibraryService.java:29--30, MultiClassLoader), so a downstream library can see and subclass arend-lib's classes: SimplificationRule is public, and LocalSimplificationRuleBase is public with a public constructor.

arend-lib is the standard library, not a privileged one.

4.2 But this tactic's rule table is closed

The narrow claim that survives is about Algebra.simplify specifically:

- Simplifier.getSimplificationRulesForType is private, a hardcoded cascade with no registry and no setter (Simplifier.java:150--205).
- StdExtension never retains the instance: new SimplifyMeta() is constructed inline inside a lambda (StdExtension.java:537). There is no handle to reach from outside.
- SimplifyMeta's @Dependency fields are package-private except GroupData and CGroupData, so an external rule cannot read meta.mul, meta.ideLeft or meta.concat.

So one cannot add a rule to simplify. One can write MySimplify beside it.

4.3 The distinction that matters: granularity

Both systems are open. What differs is the size of the smallest contributable unit, and it cuts three ways.

Grain. For this comparison Lean offers three relevant contribution units: a lemma (@[simp]), a simproc, and a tactic [5, 15, 4]. Arend has one: the meta. There is no sub-tactic unit, so the smallest contribution to simplification is an entire simplifier. @[simp] theorem foo is one line in a user's file; the Arend equivalent is a Java class, a Gradle build, and a jar in ext/.

Medium. Arend metas are JVM code in a separately compiled artefact, out of band from the .ard sources they serve. Lean metaprograms are themselves Lean definitions and can live beside the declarations they manipulate [20, 15]. The friction difference is visible in this repository's eight centrally implemented rules versus the @[simp] declarations distributed throughout Mathlib [30].

Composition — the real cost. If ten libraries each ship a simplifier meta, the user gets ten tactics and must sequence them by hand (RepeatMeta, OrElseMeta exist for exactly this). If ten libraries each ship @[simp] lemmas, the user still calls simp and gets all ten. A shared registry beats N independent tactics because rule *interaction* is where a simplifier's power comes from, and hand-composed metas cannot interleave their rules inside a single fixed-point traversal. The Lean half of this comparison follows from the import-merged simp and simproc extensions [5, 15]; the compositional conclusion is an architectural inference.

The gap is therefore not "closed versus open" but **no shared rule registry versus a shared rule registry**.

4.4 Arend already has the attribute primitive

The natural objection to the previous subsection is that a shared registry would need new language machinery. It would not. Arend already has the persistence-and-hook half of an attribute system, and arend-lib uses it in production today.

`@Dependency` is an attribute pointed the other way. Compare the two directions:

```
@Dependency(name = "Monoid.ide-left") ArendRef ideLeft;   // the tactic names the lemma
```

This is a *pull*, resolved and checked at extension load. The tactic author must enumerate everything the tactic will ever use. An attribute is a *push*: the declaration announces itself into a table, and the tactic consumes whatever is present — without the tactic's author knowing in advance what will appear. That inversion, not permission, is the whole of the extensibility difference.

The primitive exists. Every component needed by Lean's *simp-related* attribute mechanism has a counterpart already present in the Arend extension API [2, 5, 15]:

Lean	Arend counterpart	Location
environment extension	`SerializableKey<Data>` with `serialize` / `deserialize`	`FieldKey.java:25,44,60`
persistence across imports	metadata written into the compiled binary	*ibid.*
attribute application	`putUserData(key, data)` on the definition	`FieldKey.java:82`; `api/.../UserDataHolder.java:8`
declaration-time hook	`DefinitionListener.typechecked(CoreDefinition)`	`api/.../DefinitionListener.java`
attribute registration	`registerKeys` → `registerAllKeys(this)`	`api/.../ArendExtension.java:22`

`StdExtension` already runs three such keys (`StdExtension.java:60--66`):

```
public final IrreflexivityKey irreflexivityKey = new IrreflexivityKey("irreflexivity", this);
public final TransitivityKey  transitivityKey  = new TransitivityKey("transitivity", this);
public final ReflexivityKey   reflexivityKey   = new ReflexivityKey("reflexivity", this);

private final ListDefinitionListener definitionListener =
  new ListDefinitionListener().addDeclaredListeners(this);
```

Persistent, import-surviving, per-definition metadata populated by a hook that fires on every typechecked definition: that is an attribute system.

What is missing is the opt-in. The existing keys do not let an author tag anything. `FieldKey.typechecked` *infers* eligibility from the shape of the definition (`FieldKey.java:118`):

```
public void typechecked(@NotNull CoreDefinition definition) {
  if (!(definition instanceof CoreClassDefinition classDef)) return;
  if (!Names.isSetHierarchy(classDef)) return;
  for (CoreClassField field : classDef.getPersonalFields()) {
    if (checkField(field)) break;
  }
}
```

For `IrreflexivityKey`, `checkField` tests whether the field is a binary relation into `Empty` over the carrier. Nobody tags anything: this is **attributes by structural inference** rather than attributes by annotation.

That is a genuine design difference rather than merely a missing feature. Inference means a library author cannot decline, and cannot express an intent that is not already visible in the type. But it also means the plumbing a simplification-rule registry would need is built and in

service; the only open question is how a rule announces itself — structurally, as `FieldKey` does, or through some marker a listener recognises.

Consequence for the registry. A shared rule table for `simplify` is therefore not a language-level request. It can be built on `SerializableKey` plus `DefinitionListener` — which additionally buys import-merging and persistence, making it a true analogue of `register_simp_attr` rather than a hand-rolled substitute [5] — and consumed where `getSimplificationRulesForType` currently hardcodes its cascade (`Simplifier.java:150`). Third-party population would go through `setDependencies` (Section 4). The `@Dependency` mechanism is orthogonal and carries over unchanged.

5 Case study: `field_simp`

`field_simp` clears denominators, turning $a/b + c/d = e$ into a polynomial identity that `ring` can finish. It is the natural next target for an Arend simplification rule, and its history in Lean is directly instructive: the Lean 3 implementation was a simp-rule set whose documentation notes that it did not detect common denominator factors; current Mathlib uses a typed normalising procedure with an explicit gcd operation [21, 22]. The redesign pull request describes the change as a rewrite for robustness and performance and reports a representative reduction from 19,983 to 2,979 heartbeats [23]. The asymptotic cost analysis below remains an inference made in this note rather than a result measured by that pull request.

5.1 The first design: a simp set plus a discharger

In its Lean 3 form the tactic was a `simp` call [21]:

```
field_simp [hx, hy]
-- was shorthand for
simp [-one_div, -mul_eq_zero, hx, hy] with field_simps
  { discharger := tactic.field_simp.ne_zero }
```

Reading the expansion. Each part configures one piece of the generic Lean 3 simplifier:

- `field_simp [hx, hy]` is the user-facing command. The bracketed arguments are not denominators to clear; they are proofs or rewrite lemmas supplied for this invocation. Typical examples are `hx : x ≠ 0` and `hy : y ≠ 0`.
- `simp [...] with field_simps` runs the ordinary simplifier with its default rules, the explicitly listed arguments, and the additional theorem set tagged `field_simps`. That set contains the domain-specific rules which move divisions through products and sums and ultimately clear denominators in a relation.
- The leading minus in `-one_div` removes the default `one_div` simp lemma for this call. That lemma rewrites in the direction opposite to the field-specific scheme, which first represents inverses as divisions; leaving both directions enabled would work against the algorithm. The implementation also removes `one_divp`, the analogous rule for partial division by units; the published short form above omits that extra case.
- Likewise, `-mul_eq_zero` removes the default rewrite of a zero product into a disjunction. The field-specific set uses the conditional `mul_ne_zero` rule instead, turning a product-nonzero condition into two smaller nonzero conditions that the discharger can attempt separately.

- `hx` and `hy` add the caller's facts to the simp set. The final configuration record selects `tactic.field_simp.ne_zero` as its `discharger`, overriding simp's side-condition solver. Whenever a conditional field rule requires a proof such as `d` $\neq$ `0`, this tactic first tries `assumption` and, when `d` is numerical, `norm_num1`. If neither succeeds, the conditional rewrite is skipped and the corresponding denominator may remain.

Thus the old tactic was not a separate normalisation engine: it was a particular rule set, two removals from the ambient simp set, user-supplied facts, and a specialised callback layered on top of `simp` [21].
The mathlib3 documentation describes the algorithm as naive: it does not detect common factors between denominators, instead leaving the more complicated resulting expression to `ring` [21].
Two costs follow. First, denominator blowup is by design: cross-multiplying k distinct denominators yields a numerator of $O(k)$ terms of $O(k)$ factors, so term size grows as $O(k^2)$ before `ring` even starts. Second, the discharger. Because `norm_num` cannot operate on disjunctions, `field_simp` suppresses `mul_eq_zero` in favour of `mul_ne_zero : x` $\neq$ `0` $\to$ `y` $\neq$ `0` $\to$ `x * y` $\neq$ `0`, so every nested product denominator spawns a *tree* of side goals, each invoking `norm_num`. This is why the documentation advises passing hypotheses explicitly. This cost analysis is derived from the documented rewrite scheme; the disjunction and nonzero-product behaviour is stated directly in the same source [21].

5.2 The new normaliser and its `simp [field]` front end

First, the central distinction. An ordinary simp lemma is a theorem that says in advance how one form may be rewritten. A *simp procedure*, or *simproc*, is executable Lean code: when `simp` reaches an expression of a registered shape, it calls the procedure. The procedure may inspect the whole expression, compute a replacement, and return a proof that the replacement is valid; it may also decline to change anything [15].

Mathlib exposes its field normaliser to `simp` through the named set `field`. Consequently, `simp [field]` does *not* attack every division with a separate local rewrite. When `simp` visits an equality or inequality, it can hand that whole comparison to `FieldSimp.proc`. The procedure normalises both sides together and clears or cancels only those denominators for which it can establish the required side conditions [22].

Here is the paper-and-pencil picture. Suppose the surrounding context contains `hx : x` $\neq$ `0` and `hy : y` $\neq$ `0`. On reaching

$$\frac{a}{x} + \frac{b}{y} = c,$$

the procedure can return a denominator-free comparison schematically like

$$ay + bx = cxy.$$

This display explains the algebra, not the exact term that Lean prints: the implementation chooses a canonical ordering and may cancel shared factors. Crucially, this is one certified transformation of the *whole equality*, not a sequence of unrelated rewrites at the two division signs.

activate `field` $\longrightarrow$ match a comparison $\longrightarrow$ build a normal form $\longrightarrow$ prove denominator conditions $\longrightarrow$ return a checked comparison $\longrightarrow$ continue ordinary simplification

More precisely, a call proceeds as follows.

1. `simp [field, hx, hy]` enables the named `field` set and also makes the supplied facts available to the surrounding simplifier.
2. When `simp` visits a subexpression whose outer form is `Eq`, `LE.le`, or `LT.lt`—Lean's internal names for $=$, $\leq$, and $<$—it selects `FieldSimp.proc`. The comparison may itself be nested inside a larger expression; "whole comparison" means the entire currently matched $=$, $\leq$, or $<$ node.
3. The procedure converts the two sides to `qNF` values. A `qNF` represents a product of distinct, consistently ordered *atoms* raised to integer powers. An atom is simply a subexpression treated as an opaque variable; a negative power records a denominator.
4. On this structured representation the procedure merges equal atoms, combines their powers, and extracts factors already shared by the two sides. When it considers such an extraction, a `DenomCondition` specifies the kind of certificate needed: none, nonzeroness, or positivity.
5. The discharger tries to construct that certificate from assumptions and arithmetic procedures. Nonzeroness justifies cancellation; positivity may be needed when clearing a factor from an inequality so that its direction is preserved. If the required justification cannot be produced, the procedure declines that transformation rather than making an unsafe rewrite.
6. The procedure returns both the transformed comparison and a proof that it is equivalent to the original one. The result `.visit` asks ordinary `simp` to visit the new expression; `.continue` means that the procedure made no change and simplification should continue normally.

The remaining source declarations implement that story.

The three triggers.

```
simproc_decl fieldEq (Eq _ _)    := FieldSimp.proc
simproc_decl fieldLe (LE.le _ _) := FieldSimp.proc
simproc_decl fieldLt (LT.lt _ _) := FieldSimp.proc

attribute [field, inherit_doc FieldSimp.proc] fieldEq fieldLe fieldLt
```

Each pattern is a lookup key telling `simp` when to try the callback. There is deliberately no trigger for `_ / _` or `_ + _`: all three relation shapes call the same whole-relation procedure. The last line registers the three declarations in the named `field` set; `inherit_doc` only shares their documentation text [22, 15].

The computed normal form. The relevant data have a small conceptual interface:

```
abbrev qNF -- sorted entries: (integer power, quoted atom, atom index)

inductive DenomCondition | none | nonzero | positive
```

Each entry is really of shape `((power, atom), index)`. The index identifies the atom and sorts the list; entries with the same index denote the same atom. Thus multiplication and division can merge two lists much like sparse exponent maps, adding or subtracting powers when their indices agree. Proof-producing functions then reconstruct a Lean expression and certify the computation. The implementation uses Qq typed quotations, also used by Mathlib's `ring` normaliser: `Q(T)` tracks that a generated core expression has Lean type `T`, and Lean's kernel still checks every produced proof [22, 28].

In particular, `qNF.gcd` compares two sorted factor lists and extracts powers of atoms represented in both, provided the required certificate can be built. It is not a Euclidean gcd or a

general polynomial factoriser. This ability directly addresses a limitation documented for the mathlib3 tactic, which did not detect shared denominator factors; the redesign also targeted robustness and performance [21, 22, 23].

`DenomCondition` is not an extra field stored in every `qNF`. Instead, it selects the type of certificate accompanying an extracted factor: trivial evidence for `none`, a proof that the evaluated factor is nonzero, or a proof that it is positive. Thus `qNF.gcd` returns both the common factor and the justification needed to use it safely [22].

How `simp` discovers it. `Mathlib/Tactic/FieldSimp/Attr.lean` creates the named registry at module initialisation:

```
initialize fieldSimpExt : Simp.SimprocExtension <-
  Simp.registerSimprocAttr `field ... none
```

`Simp.SimprocExtension` means that this import-aware registry holds procedures, not a collection of rewrite theorems. Here `field` is only the registry's name; it is unrelated to Lean's algebraic `Field` type class. The final `none` says to start this registry empty instead of seeding it from an existing built-in registry. This procedure-only restriction applies to the named `field` set, not to the surrounding simplifier: ordinary `[simp]` lemmas still participate when `simp [field]` runs [8, 15, 24].

How control returns to `simp`. Internally, `FieldSimp.proc` reads the surrounding `Simp.Context`, uses `FieldSimp.discharge`, and calls `reduceProp` to normalise the matched relation. Before returning, it runs a restricted cleanup with `simpOnlyNames [``one_div, ``mul_inv]`. This *applies* those two lemmas to put any residual inverse/division syntax into the form expected by ordinary `simp`; it does not remove the lemmas. Lean composes the cleanup proof with the normalisation proof, then returns `.visit`. Keeping this boundary consistent prevents the field normal form and the ambient simp normal form from rewriting against one another in a loop [22].

The standalone `field_simp` tactic and `simp [field]` are therefore two front ends, not two algebraic algorithms. The former calls the normalising core directly and recursively; the latter reaches `FieldSimp.proc` through the simplifier's named-set lookup. Both use the same normal-form and discharger machinery [22].

In one sentence: `simp` is now the dispatcher, while `FieldSimp.proc` is the proof-producing algebra engine for an entire comparison.

5.3 The discharger

`Mathlib/Tactic/FieldSimp/Discharger.lean`:

```
partial def discharge (prop : Expr) : SimpM (Option Expr)
```

That is the `Simp.Discharge` shape, which is why one discharger serves both the simproc and the standalone tactic. Four strategies, in code order:

1. `Simp.dischargeUsingAssumption? prop`;
2. `NormNum.derive`, matched on `~q(($e : $α) ≠ $b)`, keeping the proof only on `.isTrue`;
3. `Positivity.solve`;
4. recursive `simp` under `Simp.withIncDischargeDepth`, with the ambient context augmented by

```
[``two_ne_zero, ``three_ne_zero, ``four_ne_zero, ``mul_ne_zero,
 ``pow_ne_zero, ``zpow_ne_zero, ``Nat.cast_add_one_ne_zero]
```

added with `post := false`, and passing `discharge` itself back in as the discharger. Reaching `True` yields `mkOfEqTrue`.

`Simp.withIncDischargeDepth` is the termination guard. The discharger is exposed for debugging as `field_simp_discharge` through `wrapSimpDischarger`; the wrapper exists because `SimpM` has no `Term.State`, so info-tree updates require an `IO.Ref`. The four strategies are implemented in the cited discharger module; the wrapper and its state bridge have their own utility module, and the depth guard is part of Lean's generic simp API [25, 26, 17, 16].

5.4 Two front ends over one core

The standalone tactic bypasses `simp` entirely:

```
elab (name := fieldSimp) "field_simp" d:(discharger)? args:(simpArgs)? loc:(location)? :
    tactic => withMainContext do
  let disch <- parseDischarger d args
  let s <- IO.mkRef {}
  let cleanup r := do r.mkEqTrans (<- simpOnlyNames [] r.expr)
  let m := AtomM.recurse s { contextual := true } (wellBehavedDischarge := false)
    (fun e => reduceProp disch e <|> reduceExpr disch e) cleanup
  let loc := (loc.map expandLocation).getD (.targets #[] true)
  transformAtLocation (m .) "field_simp" (ifUnchanged := .error)
    (mayCloseGoalFromHyp := true) loc
```

`AtomM.recurse`, not `simp`. The simproc is the *second* front end: it exists so that `simp`, and everything built on `simp`, picks up field normalisation when its traversal reaches an `Eq`/`LE`/`LT` node. Setting `wellBehavedDischarge := false` tells the recursive traversal that discharge may depend on newly introduced local declarations, so its simp results must not be cached as aggressively [22, 29].

6 Cost model for a local-rule implementation in Arend

Suppose one implemented `field_simp` as a `LocalSimplificationRuleBase` subclass. What would it cost? The following is derived from the code, not measured; the distinction is maintained throughout.

6.1 Per-step cost

`LocalSimplificationRuleBase.apply` (line 42) is a fixed-point loop whose body pays, for a term of n core nodes:

1. A full traversal from the root, **restarted after every single rewrite**: the visitor returns `STOP` on the first hit (line 69) and the enclosing `while (true)` re-traverses from scratch.
2. `subexpr.computeType().compare(expression.getType(), CMP.EQ)` on *every visited node* (line 50) — a type computation plus a unifier call each. `CoreExpression.computeType()` carries no caching guarantee in the API (`api/.../CoreExpression.java:34`).
3. `typechecker.typecheck(lam, null)` on `\lam i => ⟨entire term with p @ i spliced in⟩` (line 122): a full elaboration of an $O(n)$ term.
4. Two `replaceSubexpressions` passes (lines 104, 115), plus `typechecker.check(simplifiedExpr, refExpr)` (line 125).
5. On return, `Simplifier.java:117` elaborates the result *again*.

So roughly **three full elaborations plus $O(n)$ type-compares per rewrite step**.

6.2 Why this is fine today and would not be for `field_simp`

The eight existing rules are unaffected by this cost model because every one of them **strictly contracts** the term: $x \cdot 1 \to x$, $x \cdot 0 \to 0$, $-(-x) \to x$. n is small and shrinking, so re-elaboration is a constant factor on a tiny input.

`field_simp` is the opposite: it **expands**. Combining k fractions takes the term from $O(k)$ to $O(k^2)$, and intermediate terms grow monotonically through the rewrite sequence. The re-elaboration cost is therefore paid repeatedly on a growing input — which is exactly the regime the current design never encounters.

6.3 Estimated knee

Estimate, not measurement.

Denominators k	Nodes n	Steps	Expectation
3	∼30	∼5	negligible
5	∼75	∼8	single-digit seconds; annoying but usable
10	∼200–300	∼20	10^4 unifier calls, ∼60 elaborations of 300-node terms

The knee is somewhere around five to eight distinct denominators. Two caveats in the honest direction: if `computeType` is cheap on already-typed core expressions, or if `compare` short-circuits on normalised types, the knee moves right, possibly far. I cannot say how far without measuring.

6.4 The blocker is not performance

Arend's field is no kinder than Lean's. `arend-lib/src/Algebra/Field.ard:50`:

```
| finv-right {x : E} (x/=0 : x /= 0) : x * finv x = 1
```

Every cancellation is gated on `x /= 0`. Now recall the hook one would be implementing:

```
protected abstract Pair<CoreExpression, ConcreteExpression>
  simplifySubexpression(CoreExpression subexpr);
```

There is no way for that signature to say *"I can rewrite this if someone proves $b \neq 0$"*. No discharger hook, no subgoal channel, no depth budget, nothing analogous to `Simp.Discharge`. One would hand-roll a context search inside each rule — and because rules fire per subexpression inside a restarted traversal, that search would be redone at every step, reproducing the mathlib3 discharger-tree cost with none of Lean's infrastructure for bounding it [21, 17, 6].

6.5 The same hole, seen from the congruence side

The missing discharger is one face of a more general absence. Item 4 of Section 2.4 — contextual information — is the other.

Lean exposes it as a simp configuration flag, and the `field_simp` frontend switches it on explicitly:

```
AtomM.recurse s { contextual := true } (wellBehavedDischarge := false) ...
```

Contextual rewriting means a hypothesis available *at a position* may be introduced while descending through a binder or congruence rule and then used when rewriting there: `c` inside the `then` branch, a $\neq$ `0` fact inside the scope that established it. Denominator nonzero-ness is contextual in exactly this sense [6, 29, 22].

Arend's `simplify` has nowhere to thread such a fact. Neither mechanism it uses provides a slot: `pmap`-style congruence substitutes into a context without consulting it, and the top-level

`transport` chain abstracts occurrences out of the type before any of them is examined. This is the same gap as the missing discharger channel above, approached from the congruence side rather than the rule-interface side.

It is also a further argument for the design of Section 7: gathering the `/= 0` facts once, at the root, as explicit hypotheses of a correctness lemma sidesteps positional context entirely. One cannot thread contextual information through a traversal that has no channel for it — but one can decline to need it.

7 Recommended design

Follow `GroupInverseRule`, not `LocalSimplificationRuleBase`. The repository already contains the pattern end to end.

1. **Term AST.** Define `FieldTerm` mirroring `GroupTerm` (`Solver.ard`), with constructors for atoms, product, sum, inverse and possibly integer powers. Compile core expressions into it with `CompiledTerm.compile` and instance-aware `FunctionMatchers`, as `GroupRuleBase` does.

2. **Normal form.** Normalise in Java to a $(num, denom)$ pair — one global pass, no intermediate elaboration. Borrow `qNF`'s shape: a list of (atom, integer exponent) with a canonical atom order makes cancellation and gcd extraction straightforward [22].

3. **Side conditions as data.** Adopt `DenomCondition`'s idea: track per denominator whether nonzero-ness is unknown, known, or known positive, and carry it with the normal form. Do not attempt to discharge during normalisation [22].

4. **One correctness lemma.** Prove in Arend, in the shape of `NatData.simplify-correct` (`Solver.ard:507`), something like

   ```
   \lemma field-normalize-correct (t : FieldTerm V)
                                  (nz : DList (\lam d => interpret d /= 0) (denominators t))
     : interpret t = interpret (normalize t)
   ```

   taking the nonzero facts as explicit hypotheses. Emit a single application of it. Relative to Lean's recursive discharger, this design gathers the `/= 0` proofs once, up front, instead of via a `mul_ne_zero` goal tree. This is a proposed tradeoff, not a measured performance result [25].

5. **Fire at the relation, not at subterms.** The Arend analogue of `simproc_decl fieldEq (Eq _ _)` is a rule that runs once at the root of the equality. `Simplifier.simplifyTypeOfExpression` already hands the type over before descending, so intercept there rather than as a per-subexpression rule [22].

6. **Register the classes.** Add `Field` / `DivisionRing` to `possibleClasses` and the cascade (`Simplifier.java:153`). Because `getSimplificationRulesForType` is `private`, this lands as an `arend-lib` patch either way (Section 4).

7. **Optionally, take the registry detour first.** If this rule is likely to be the first of several, consider replacing the cascade with a `SerializableKey`-backed rule table before adding to it (Section 4.4). The cost is bounded — the persistence and hook machinery already exists and is in service for `irreflexivityKey` and friends — and it converts every subsequent rule from an `arend-lib` patch into an ordinary library contribution.

7.1 Cheap way to falsify the estimate first

Before building any of the above, write the dumbest possible local rule for $a/b + c/d \to (ad + bc)/(bd)$ alone, with the nonzero proofs passed in by hand so the discharger problem is

sidestepped entirely, and time it at 3, 6 and 10 denominators. If the curve is flat, Section 6 is wrong and the local route is viable. If it is superlinear, the answer is settled in an afternoon rather than after a full implementation.

8 Incidental defects observed

Nondeterministic rule ordering. `Simplifier.java:78`:

```
var simplificationRules = new TreeSet<SimplificationRule>(
  (o1, o2) -> o1.equals(o2) ? 0 : o1.hashCode() - o2.hashCode());
```

`MonoidIdentityRule` overrides `equals` but not `hashCode`, so ordering falls back to identity hash codes and varies between JVM runs; the subtraction can also overflow. Deduplication still works, since the comparator returns 0 on `equals`. This is harmless while all rules are contracting and confluent, and becomes a reproducibility bug the moment a non-confluent rule — such as anything `field_simp`-shaped — is added.

Dead code. `Simplifier.java` carries four commented-out blocks (lines 88–93, 125–128, 220–223, 265–288). The first is an abandoned attempt to gather rules from *subexpression* types rather than the root type, which is exactly the generalisation a pattern-indexed engine would need.

9 Summary

Arend's `simplify` is a well-factored tactic with a hardcoded rule table. Lean's `simp` is an open rewriting platform. The traversal, fixed-point iteration and proof-chaining machinery correspond closely; what is absent is the three things that make `simp` a platform: a rule registry a declaration can announce itself into, head-pattern indexing so lookup scales with rule count, and core-term rather than surface-syntax rule output [11, 9, 16, 15].

Of those three, the first is the least distant. Arend has the persistence and declaration-hook machinery already — `SerializableKey` plus `DefinitionListener`, in service today for `irreflexivityKey` and its siblings — and what it lacks is the opt-in: existing keys infer eligibility from a definition's shape instead of letting an author declare it (Section 4.4). The registry is a library-level change, not a language-level one.

For `field_simp` specifically the conclusion is sharper. Lean 3's documented rewrite-rule design did not detect common denominator factors; current Mathlib uses a global normaliser with `qNF.gcd`, exposed through simprocs keyed on relations rather than arithmetic [21, 22]. The redesign was explicitly presented as improving robustness and performance [23]. Arend should start with reflection, one correctness lemma, and side conditions carried as data.

10 Repository source locations

This repository.

- `arend-lib/meta/src/main/java/org/arend/lib/meta/simplify/` — the framework.
- `arend-lib/meta/src/main/java/org/arend/lib/StdExtension.java:537` — registration.
- `arend-lib/src/Algebra/Group/Solver.ard:504,507,576` — reflective normaliser and its correctness proofs.
- `arend-lib/src/Algebra/Field.ard:48--50` — `finv` and `finv-right`.
- `arend-lib/src/Paths.ard:3` — `pmap`, i.e. congruence as a definition.

- `arend-lib/meta/src/main/java/org/arend/lib/meta/cong/CongruenceMeta.java` — congruence *closure*, not `@[congr]`.
- `arend-lib/test/Algebra/SimplifyTest.ard` — the 25 regression tests.
- `api/src/main/java/org/arend/ext/ArendExtension.java` — extension interface.
- `base/src/main/java/org/arend/server/impl/LibraryService.java:29--30,158` — extension loading and classloader tiers.
- `arend-lib/meta/src/main/java/org/arend/lib/key/FieldKey.java` — the `SerializableKey` + `DefinitionListener` pattern; subclasses `IrreflexivityKey`, `TransitivityKey`, `ReflexivityKey`.
- `api/src/main/java/org/arend/ext/typechecking/DefinitionListener.java`, `api/src/main/java/org/arend/ext/userData/UserDataHolder.java`, `api/src/main/java/org/arend/ext/serialization/SerializableKey.java` — the attribute primitive.

References

[1] Lean Project, *The Lean Language Reference: Attributes*, accessed 31 August 2026. `https://lean-lang.org/doc/reference/latest/Attributes/`

[2] Lean Project, *The Lean Language Reference: Elaboration and Compilation*, accessed 31 August 2026. `https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/`

[3] Lean Project, *The Lean Language Reference: The Simplifier*, accessed 31 August 2026. `https://lean-lang.org/doc/reference/latest/The-Simplifier/`

[4] Lean Project, *The Lean Language Reference: Invoking the Simplifier*, accessed 31 August 2026. `https://lean-lang.org/doc/reference/latest/The-Simplifier/Invoking-the-Simplifier/`

[5] Lean Project, *The Lean Language Reference: Simp sets*, accessed 31 August 2026. `https://lean-lang.org/doc/reference/latest/The-Simplifier/Simp-sets/`

[6] Lean Project, *The Lean Language Reference: Configuring Simplification*, accessed 31 August 2026. `https://lean-lang.org/doc/reference/latest/The-Simplifier/Configuring-Simplification/`

[7] Lean Project, *Lean 4.0.0-m4 release notes*, 27 March 2022. `https://lean-lang.org/doc/reference/latest/releases/v4.0.0-m4/`

[8] Lean Project, *Lean 4.6.0 release notes*, 29 February 2024. `https://lean-lang.org/doc/reference/latest/releases/v4.6.0/`

[9] Lean Project, `Lean/Meta/DiscrTree/Main.lean`, Lean 4.32.1: discrimination tree representation, reduction and arity/skip rationale. `https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Meta/DiscrTree/Main.lean`

[10] Lean Project, `Lean / Meta / DiscrTree / Types . lean`, Lean 4.32.1: `DiscrTree.Key` constructors. `https : / / github . com / leanprover / lean4 / blob / f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Meta/DiscrTree/Types.lean`

[11] Lean Project, `Lean/Meta/Tactic/Simp/SimpTheorems.lean`, Lean 4.32.1: simp-theorem records, priorities and discrimination-tree index. `https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Meta/Tactic/Simp/SimpTheorems.lean`

[12] Lean Project, Lean/Meta/CongrTheorems.lean, Lean 4.32.1: CongrArgKind, CongrTheorem and congruence generation. https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Meta/CongrTheorems.lean

[13] Lean Project, Lean/Meta/Tactic/Simp/SimpCongrTheorems.lean, Lean 4.32.1: user-defined simp-congruence theorems. https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Meta/Tactic/Simp/SimpCongrTheorems.lean

[14] Lean Project, Init/SimpLemmas.lean, Lean 4.32.1: hand-written congruence lemmas including the conditional congruence rules. https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Init/SimpLemmas.lean

[15] Lean Project, Lean/Meta/Tactic/Simp/Simproc.lean, Lean 4.32.1: simproc registration, extensions and pattern-indexed lookup. https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Meta/Tactic/Simp/Simproc.lean

[16] Lean Project, Lean/Meta/Tactic/Simp/Types.lean, Lean 4.32.1: SimpM, caches, Result, Step, simproc types and discharge depth. https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Meta/Tactic/Simp/Types.lean

[17] Lean Project, Lean/Meta/Tactic/Simp/Rewrite.lean, Lean 4.32.1: conditional rewriting and the Simp.Discharge interface. https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Meta/Tactic/Simp/Rewrite.lean

[18] Lean Project, Lean/Elab/Tactic/Simp.lean, Lean 4.32.1: tactic syntax, named theorem/simproc set resolution, simp only and custom dischargers. https://github.com/leanprover/lean4/blob/f054605aea4b840552cca2e725580bffd1e1b704/src/Lean/Elab/Tactic/Simp.lean

[19] Lean Project, *The Lean Language Reference: Congruence Closure*, accessed 31 August 2026. https://lean-lang.org/doc/reference/latest/The--grind--tactic/Congruence-Closure/

[20] Leonardo de Moura and Sebastian Ullrich, *The Lean 4 Theorem Prover and Programming Language*, CADE 2021. https://lean-lang.org/papers/lean4.pdf

[21] Mathlib contributors, *mathlib3 documentation: tactic.field_simp*, the rewrite-rule implementation and its documented limitations. https://leanprover-community.github.io/mathlib_docs/tactic/field_simp.html

[22] Mathlib contributors, Mathlib/Tactic/FieldSimp.lean, mathlib4 commit 520045a: current normal form, normaliser, tactic and relation-keyed simprocs. https://github.com/leanprover-community/mathlib4/blob/520045ab14e26149ee970e2e617ca04b09bde5d6/Mathlib/Tactic/FieldSimp.lean

[23] Heather Macbeth, *feat(Tactic/FieldSimp): rewrite for performance and robustness*, mathlib4 pull request 28658, merged August 2025. https://github.com/leanprover-community/mathlib4/pull/28658

[24] Mathlib contributors, Mathlib/Tactic/FieldSimp/Attr.lean, mathlib4 commit 520045a: the field simproc attribute. https://github.com/leanprover-community/mathlib4/blob/520045ab14e26149ee970e2e617ca04b09bde5d6/Mathlib/Tactic/FieldSimp/Attr.lean

[25] Mathlib contributors, `Mathlib/Tactic/FieldSimp/Discharger.lean`, mathlib4 commit `520045a`: nonzero/positivity discharge procedure. `https://github.com/leanprover-community/mathlib4/blob/520045ab14e26149ee970e2e617ca04b09bde5d6/Mathlib/Tactic/FieldSimp/Discharger.lean`

[26] Mathlib contributors, `Mathlib/Util/DischargerAsTactic.lean`, mathlib4 commit `520045a`: adapting a `Simp.Discharge` procedure to a tactic while preserving elaborator state through an `IO.Ref`. `https://github.com/leanprover-community/mathlib4/blob/520045ab14e26149ee970e2e617ca04b09bde5d6/Mathlib/Util/DischargerAsTactic.lean`

[27] Mathlib contributors, `Mathlib/Tactic/Attr/Register.lean`, mathlib4 commit `520045a`: central custom simp-attribute declarations. `https://github.com/leanprover-community/mathlib4/blob/520045ab14e26149ee970e2e617ca04b09bde5d6/Mathlib/Tactic/Attr/Register.lean`

[28] Mathlib contributors, `Mathlib/Tactic/Ring/Basic.lean`, mathlib4 commit `520045a`: the typed, proof-producing ring normaliser. `https://github.com/leanprover-community/mathlib4/blob/520045ab14e26149ee970e2e617ca04b09bde5d6/Mathlib/Tactic/Ring/Basic.lean`

[29] Mathlib contributors, `Mathlib/Util/AtomM/Recurse.lean`, mathlib4 commit `520045a`: recursive normalisation and contextual-discharge configuration. `https://github.com/leanprover-community/mathlib4/blob/520045ab14e26149ee970e2e617ca04b09bde5d6/Mathlib/Util/AtomM/Recurse.lean`

[30] The Mathlib Community, *The Lean Mathematical Library*, CPP 2020. `https://leanprover-community.github.io/papers/mathlib-paper.pdf`